PROJECT SENSEWALK

Project Report

Software, AI models and where things stand

A smart assistive walker for visually impaired users: what the team has built, how the two AI models were made and measured, what has been tested, and what still needs real hardware.

Repository	github.com/sayansaunth-lang/sensewalk (130 files, 24 commits)
Hardware in hand	ESP32-S3, Raspberry Pi 3B (1 GB), Pi Camera Module 3
Software	Built and tested on a laptop: 176 Python tests + 19 firmware tests, all passing; GitHub checks green
Own AI model	YOLOv8n pothole detector, 3,011,043 parameters, shipped at 6.1 MB (AP50 0.54 on 92 unseen images)
Hardware status	Walker not built. Pi not yet online; camera not yet tested. Firmware compiles but has never run on a board

How to read this report. Every number was measured on a real run and every claim that was *not* tested is labelled. Section 8 lists exactly what has and has not been verified, so nobody on the team assumes more than is true.

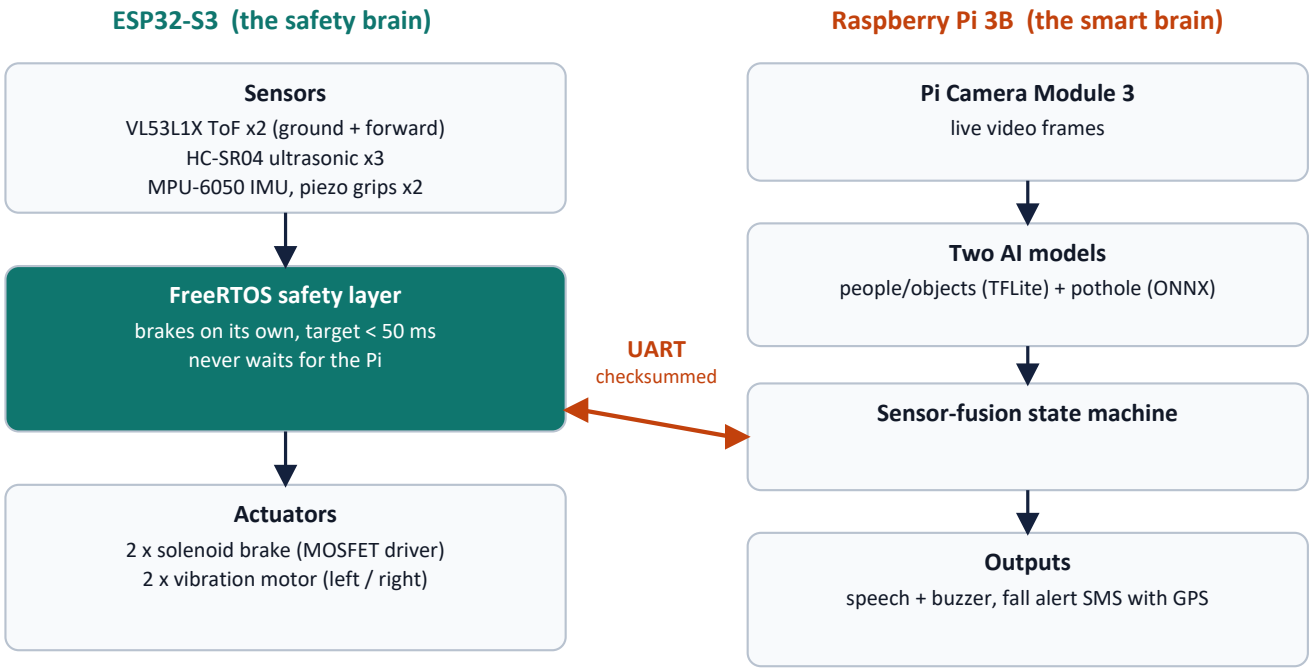
1. Where the project stands

SENSEWALK splits the work between two processors. An **ESP32-S3** senses hazards and is designed to brake by itself; a **Raspberry Pi** adds cameras, AI, speech and an emergency message. The software for both is written and tested. The physical walker, and running any of this on the real boards, is still ahead.

Area	Status	Evidence / caveat
Pi-side software (vision, fusion, speech, UART, emergency)	**Done, tested**	176 automated tests pass; GitHub runs them on every push
People / object detector (pretrained)	**Verified**	Run on real photos with real weights; finds people, makes some false detections
Pothole detector (our own model)	**Trained and compressed**	Finds about half the potholes it is shown; about 1 in 3 detections is wrong (section 4)
Fall alert: SMS with GPS location	**Built, tested on a fake modem**	Never run on a real SIM800L or GPS; needs a 2G network
ESP32 firmware (PlatformIO and Arduino IDE)	**Compiles, never flashed**	Builds on GitHub with the latest ESP32 core; 19 C++ tests pass; never run on a real board
Sensor thresholds (drop-off, obstacle, fall, grip)	**Placeholders**	Marked TODO(calibrate); must be replaced with bench measurements
Raspberry Pi 3B bring-up	**Not online yet**	Not found on any network I scanned; camera untested (section 7)
Chassis, wiring, brakes, power	**Not started**	Mechanical track

> **The most important design rule.** A camera cannot measure distance, so vision may only **warn** (speech and vibration). Only the ESP32's time-of-flight and ultrasonic sensors, which measure real distance, can fire the brake. A slow, crashed or wrong Pi therefore cannot cause a wrong brake decision.

2. How the system fits together



Why two processors? Linux on the Pi can pause for tens of milliseconds unpredictably, which a brake cannot tolerate. The ESP32 runs a real-time operating system (FreeRTOS) with the safety loop at the highest priority. The Pi has the memory and speed for cameras and neural networks, but is never trusted with the brake. The 50 ms target is a design goal: the firmware times its own decision step, but the full sensor-to-brake time still has to be measured with a logic analyser on real hardware.

The link between them

- **UART cable** at 115200 baud. The ESP32 streams distances, grip state and a heartbeat; the Pi sends haptic commands back. Every line carries a sequence number and a checksum, so corrupt or dropped data is detected instead of silently used. Both ends enforce the same field limits.
- **No dependency the other way.** If the cable is unplugged or the Pi reboots, the ESP32 keeps sensing and braking.

Team roles

Member	Role	Owns in the repository
M1	Chassis and ergonomics	`cad/`, sensor mounts, camera angle
M2	Actuation and power	Solenoid/MOSFET wiring, battery, fuse (`docs/WIRING.md`)
C1	Embedded firmware	`firmware/`
C2	Computer vision	`vision/pi/`, `training/`
C3	Integration and fusion	`comms/`, `fusion/`, `emergency/`
C4	Speech, test and docs	`speech/`, `test/`, README, demo

3. What has been built

Part	What it does	Folder
ESP32 firmware	Reads the ToF, ultrasonic, IMU and grip sensors; decides to brake on its own; drives the solenoid brake (with a 5 s anti-overheat cutoff) and vibration motors; talks to the Pi. Two copies: PlatformIO (the tested original) and a flat Arduino IDE version, plus a blink test to check your toolchain first.	`firmware/`
UART protocol	The wire format between the boards, implemented and unit tested in both Python and C++, plus a script that counts heartbeat messages from the ESP32 over a real serial port.	`comms/`
Fusion state machine	Combines sensor and vision inputs into one state (idle, walking, warning, brake engaged, fall alert) using a fixed priority order. Brake release needs a hand on the grip.	`fusion/`
Vision pipeline	Camera input (Pi Camera Module 3 or any webcam/video), the two AI models, sign reading with Tesseract, frame skipping for slow boards, and the main program that ties everything together. A simulation mode fakes the ESP32 so it runs with no hardware.	`vision/pi/`
Speech and buzzer	Short fixed alert phrases through offline text-to-speech, with a watchdog buzzer that sounds even if speech hangs.	`speech/`
Emergency alert	A detected fall starts a 10-second countdown that gripping the handle cancels; otherwise an SMS with a map link is sent, using the GPS only if its fix is recent.	`emergency/`
Model training	Dataset cleaning, YOLOv8 training, ONNX export, compression, and an evaluator that scores the deployed code path.	`training/`
Test tools	Live sensor logger with a ground-truth key, a false-positive/negative analyser, and a written field-test plan.	`test/`
Pi setup tools	One command that installs everything on the Pi and checks both models and the camera, plus an on-device speed benchmark.	`scripts/`, `vision/pi/src/benchmark.py`
Documentation	Architecture, wiring map, datasheet warnings, bill of materials, Pi 3B setup guide, a guide to every file, and this report.	`docs/`

Quality measures built in

- **Automatic checks on every push** (GitHub): the Python tests, a PlatformIO firmware build, an Arduino IDE sketch build with the latest ESP32 core, and the firmware's C++ tests.
- **Noise filtering at both layers:** a drop-off must appear in two consecutive firmware readings before braking; vision needs a detection in 2 of the last 3 inferences before warning.
- **Simulation mode** (`main.py --sim`) so the Pi side can be developed and demoed before any wiring exists.
- **Phone numbers never live in the code;** they come from an environment variable.

4. The AI models

SENSEWALK uses two neural networks and one classical text-reading engine. Only one network was trained by us. The other is already good at its job; the gap was the hazard no public model knows: potholes.

	Pothole detector (ours)	People / object detector (pretrained)
Architecture	YOLOv8n, fine-tuned from COCO-pretrained weights	SSD MobileNet v1, quantized to 8 bit
Who trained it	**We did**, on about 900 pothole photos	**Google**; we only downloaded and verified it
Parameters	**3,011,043** (about 3.0 million)	about 4.9 million 8-bit weights
File on the Pi	`hazard_yolov8n_256_fp16.onnx`, 6.1 MB	`ssd_mobilenet_v1_coco_quant.tflite`, 4.2 MB
Input size	256 x 256	300 x 300
Finds	potholes (one class)	people and 89 other everyday objects
Runs through	OpenCV (nothing extra to install)	TFLite runtime
Used for	**Warning only**	**Warning only** (person nearby)

4.1 How the pothole model was made

- **Data:** two public pothole datasets (CC BY 4.0 and MIT listings), cleaned and merged. Of 1,015 images, 913 were kept after 101 exact duplicates and a small number of unusable images were removed. Split: **730 train / 91 validation / 92 test** (1,959 labelled potholes in total).
- **Careful about leakage.** Public datasets contain several augmented copies of one photo. If one copy is in training and another in testing, the score is inflated. Images were clustered by visual similarity and each cluster kept in a single split.
- **Bad data was removed.** One source had non-label text, a personal letter, inside a label file. It was dropped and never copied into the repository.
- **Training:** fine-tuned all layers on a laptop CPU at about 66 seconds per epoch. The run was **interrupted at epoch 24 of 60** when a session ended; the best checkpoint by validation score (around epoch 17) was used. The curve had flattened, but a full run might gain a little.

4.2 Results on 92 images the model never saw

Version	File size	AP50	Precision	Recall	Hits / false alarms / misses
Original: 320 px, full precision	12.1 MB	0.511	0.64	0.49	91 / 51 / 96
320 px, FP16 weights	6.1 MB	0.511	0.64	0.49	91 / 52 / 96
256 px, FP16 (shipped)	**6.1 MB**	**0.538**	**0.68**	**0.49**	**92 / 44 / 95**
224 px, FP16	6.1 MB	0.542	0.67	0.49	92 / 45 / 95

These scores come from the code that actually runs on the Pi (the ONNX file, OpenCV and our own box-decoding), not just the training tool. AP50 was identical under OpenCV 4.8.1, 4.10.0 and 5.0.0. Hits, false alarms and misses are counted at a confidence of 0.35.

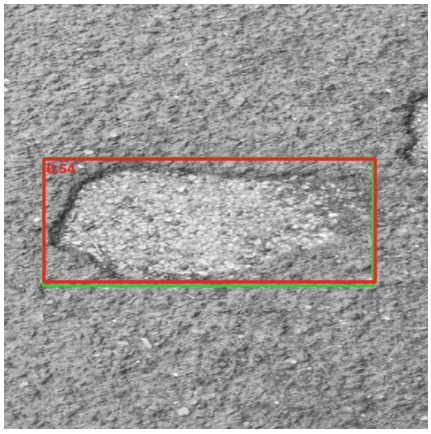
> **In plain words:** the shipped model finds about half of the potholes it is shown (recall 0.49), and about one detection in three is wrong (precision 0.68). That makes it a useful 'something odd on the ground ahead' nudge, **not** a safety device, which is why it can only trigger a spoken or vibration warning and can never trigger the brake.

4.3 Compression for the Raspberry Pi

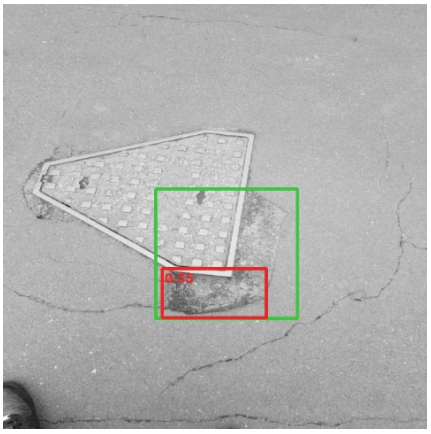
The model was compressed in two independent ways. Each was measured against the same 92 test images, after rebuilding the data split from scratch and confirming it reproduced the original numbers exactly.

- **FP16 weights halve the file** (12.1 MB to 6.1 MB) with no change in accuracy. Honest limit: OpenCV expands the weights back to 32-bit when loading, so this saves disk space and download time, **not RAM or speed**.
- **A smaller input is what makes it faster.** Going from 320 to 256 pixels does about 36% less computing. The score did not drop; it rose slightly, but with only 92 test images that is within chance, so read it as *no loss detected*, not *better*.
- **Not tried: INT8 quantization.** OpenCV cannot run the usual ONNX INT8 format and the alternatives need heavy software on a 1 GB board. Worth trying only if the Pi benchmark shows it is still too slow.

4.4 Real examples from the test set



A hit. Green is the labelled pothole, red is the model's box (confidence 0.54). They line up.



A miss. The model's box (0.35, red) covers only the lower part of the labelled pothole (green), so it does not count as a match. Scenes with covers and cracks cause errors like this.

These two were picked by rule (the first fully correct image and the first complete miss in the test set), not hand-chosen to flatter the model.

4.5 People and objects, and sign reading

- **People/objects:** checked on real photos. A street scene with a bus gave 3 people correctly plus one false 'airplane'; a photo of two people gave 5 person detections (extra boxes, so the model is noisy). To avoid false alarms, a person only counts as **nearby** if their box is at least 45% of the frame height (a crude size proxy, because a camera has no depth) and only after 2 of the last 3 frames agree. The 45% figure is a starting point to tune on real footage.
- **Sign reading (Tesseract):** runs only when a sign-like rectangle is seen. The logic is tested on synthetic images; **accuracy on real signs has not been measured.**

4.6 Known weaknesses of the pothole model

- **Small, web-sourced data** (about 900 photos, mostly daylight, from car or standing height). A walker's camera is lower and sees pavements and floors.
- **No clean-road examples.** The training photos are essentially all potholes, so cracks, shadows and manhole covers were never taught as 'not a pothole'.
- **One class only;** stairs, curbs and open drains are not detected yet. **92 test images** means true accuracy could differ by roughly 10 points either way.
- **Licence:** YOLOv8 is AGPL-3.0. Fine for a university project; decide before any distribution or commercial use.

How to improve it: film footage with the real Camera Module 3 at walker height (pavements, corridors, ramps, potholes, stairs, curbs) and label it; add hundreds of obstacle-free frames; then retrain with `training/train_hazard_model.py` and re-score with `training/evaluate_onnx.py`.

5. How the walker decides what to do

Priority	Input	Result	Can it brake?
1	Ground drop-off (downward ToF distance jumps)	Brake engaged, 'Pothole ahead, stopping'	**Yes** (real distance)
2	Obstacle close (ultrasonic or forward ToF)	Brake engaged, 'Obstacle close, stopping'	**Yes** (real distance)
3	Pothole seen by the camera	Warning, 'Hazard on the ground ahead, slow down'	No, warning only
4	Person nearby (by apparent size)	Warning, 'Person close ahead'	No, warning only
5	Sign read by OCR	Walking, speaks the text	No, information
Always first	Fall detected (IMU)	Fall alert countdown, then SMS	-

- **Brake release needs a hand on the grip.** The brake does not let go just because a hazard cleared; the piezo grip sensors must confirm the user is holding on.
- **The ESP32 applies its own copy of the brake rules directly,** so the Pi's version only drives alerts, not the safety path.
- **Alerts are short fixed phrases,** so speech is fast and predictable, and a piezo buzzer fires on its own thread if speech stalls.

Fall alert: how an SMS is sent

- A detected fall starts a **10-second countdown** ('Hold the handle to cancel the alert'). Gripping the handle cancels it; otherwise the message goes out. This exists because the fall rule can fire on things like sitting down hard, and texting family on false alarms teaches them to ignore a real one.
- The text contains a map link **only if the GPS has a valid fix less than 2 minutes old;** otherwise it says 'location unavailable'. A stale position could send help to the wrong place.
- A failed send is retried after 30 seconds; only a successful send suppresses repeats (5 minutes). One fall cannot cause a stream of texts, and a brief network drop cannot swallow the alert.
- **Hardware warnings:** the Pi 3B has one spare serial port (taken by the ESP32), so the GPS and GSM modules need two USB-to-serial adapters (not yet in the bill of materials: add them). The SIM800L is a 2G modem that needs its own 2 A supply and only works where your carrier still runs 2G.

Where the numbers come from

Every threshold (150 mm drop-off, 40 cm obstacle, grip level, fall impact) is a **starting guess** tagged `TODO(calibrate)` in `firmware/esp32/include/config.h` and `fusion/state_machine.py`. The learning roadmap requires replacing them with measurements (50+ test walks, 15+ controlled falls). Until then the hazard behaviour is untuned and no accuracy claim about it can be made.

6. Testing and quality

What	How	Result
Pi-side logic (protocol, fusion, speech, vision, emergency, training tools)	176 automated tests on a laptop, re-run on a clean Linux machine by GitHub	All pass
Firmware logic and UART codec	19 C++ tests run by GitHub on every push	All pass; the C++ checksum matches the Python one
ESP32 firmware compiles	GitHub builds it with PlatformIO and, separately, both Arduino IDE sketches with the latest ESP32 core	Passes (after 2 compile errors were found and fixed)
Pothole model	92 held-out images, OpenCV 4.8.1 / 4.10 / 5.0	AP50 0.538, identical across versions
Whole chain on real photos	Photo to detectors to confirmation filter to fusion to spoken phrase	Pothole photos say 'Hazard on the ground ahead'; people photos say 'Person close ahead'; a blank frame stays silent

Problems found and fixed while verifying

Problem	How it was found	Fix
No GPS/GSM code existed although fall-to-SMS is a headline feature	Searching the code before reporting completeness	Built the `emergency/` module
Pipeline crashed with OCR on when Tesseract was missing	Ran the real `main.py` over a video	Error handled; OCR turns itself off with one warning
First sign read would have crashed	Reading the call path against the phrase table	Pass the OCR text; regression test
A fall would latch the Pi in the fall state forever	Tracing the message flow end to end	Time-limited latch before the countdown
Any person at any distance triggered warnings	Reviewing output on real photos	Size-based 'nearby' test plus 2-of-3 confirmation
SMS driver sent the message body after the modem said ERROR	A unit test against a scripted fake modem	Check for the '>' prompt first
Firmware did not compile (struct initialisation; missing include)	First GitHub build of the firmware	Fixed in both firmware copies
UART encoder and decoder disagreed on length limits; checksum could over-read	A failing C++ test on GitHub	Both ends enforce the same limits; tests on both sides
The compressed model would have been left out of the repository	Checking what git tracked before committing	Fixed the ignore rule; confirmed the file is on GitHub
Downloads arrived corrupted on this network (twice)	pip's hash check failed	Resumable downloads with SHA-256 verification; checksums pinned in scripts

7. Raspberry Pi bring-up: honest status

The goal was to install the software on the Pi, test the Camera Module 3, and run the benchmark. **That has not happened yet, because the Pi has not come online where I could reach it.**

Step	What happened
Writing the SD card	The Imager first failed with a temporary DNS error downloading the operating system. Later you reported it finished with the 'Done' screen
Pi powered on	You reported the green light blinking, which means it started reading its card. That alone does not prove it joined Wi-Fi
Searching for the Pi	I scanned the laptop's network repeatedly: a hostel Wi-Fi (about 50 to 95 devices), your phone hotspots, 2.4 and 5 GHz. No Raspberry Pi hardware address appeared, `raspberrypi.local` never resolved, and no device had SSH open that was a Pi
Raspberry Pi Connect	Signed in to your account several times; it never listed a device
Result	The Pi is not on the network the laptop was using. **The camera and the Pi software setup are still untested.**

Likely causes, in order

- The SD card was written with a different Wi-Fi name or password from the network the laptop was on. The Pi joins **only** the network written on its card.
- The Pi 3B supports **2.4 GHz only**; hostel Wi-Fi and some hotspots use 5 GHz, and hostel networks also often need a web login and block devices from seeing each other.
- The laptop kept switching between networks, so the two devices were never on the same one.

What to do next (you, at the Pi)

- Turn on one phone hotspot at **2.4 GHz**, connect the laptop to it, and rewrite the SD card with that hotspot's exact name and password, Wi-Fi country **India**, SSH on, Raspberry Pi OS Lite (64-bit). Check the Imager's storage list shows the small SD card, **never the 476.9 GB laptop drive**.
- Or connect the Pi to a router with an **Ethernet cable**, which needs no Wi-Fi settings.
- Or plug in a screen and keyboard, log in, and run these two commands, then send me the output:

```
rpicam-hello --list-cameras
curl -fsSL https://raw.githubusercontent.com/sayansaunth-lang/sensewalk/master/scripts/pi_setup.sh | bash
```

The setup script has been syntax-checked only; its first real run will be on your Pi, so tell me any error it prints. Afterwards, measure real speed with `python3 vision/pi/src/benchmark.py`. On this laptop both models together take roughly 30 ms; a Pi 3B will be much slower, and that gap has not been measured.

8. What was tested and what was not

Not verified (do not assume these work)

Item	Why it matters
Anything running on the Pi 3B	Camera Module 3 capture, real speed, heat, text-to-speech audio, buzzer, memory use on 1 GB
ESP32 firmware on a real board	It compiles, but flashing, sensor start-up and the real-time timing have never been seen
All physical sensors and actuators	ToF addressing, ultrasonic, IMU, grips, the solenoid brake, the vibration motors
Brake within 50 ms	Only the software step is timed; needs a logic analyser or oscilloscope on the real brake
UART link between the two boards	The heartbeat-counting script exists but has never been run against a real board
Fall alert on real hardware	Real GPS fix, real SIM800L SMS, and your SIM working on 2G; tested only against a fake modem
Sign reading on real signs	Only the surrounding logic is tested
Pothole model on the walker's own camera	Trained on web photos; behaviour on your footage is unknown

9. Recommended next steps, in order

1. **Get the Pi online** (section 7), then run the setup script and the camera test `rpicam-hello --list-cameras`.
2. Run `benchmark.py` on the Pi and use the frame-skip value it suggests. Report the real numbers, even if they are low.
3. Flash the ESP32 blink test, then the full firmware with no sensors attached, to confirm it halts with a clear message when the ToF sensors are missing.
4. Wire the ToF and ultrasonic sensors and run the bench logger. Replace the placeholder thresholds with measured ones.
5. Order and wire the brake driver, then prove the under-50 ms stop with the Pi disconnected, using a logic analyser or scope.
6. Test your SIM on 2G, buy two USB-to-serial adapters, and run `python -m emergency.send_test_sms` outdoors.
7. Start filming real pothole, step and curb footage with the actual camera now. Labelled data is the slowest part of improving the model.

Appendix - Repository, licences and sources

Item	Detail
Repository	github.com/sayansaunth-lang/sensewalk. A guide to every file is at `docs/FILE_GUIDE.md`
People/object model	Google TensorFlow Lite example (coco_ssd_mobilenet_v1_1.0_quant), Apache-2.0
Pothole base network	Ultralytics YOLOv8n, COCO-pretrained, AGPL-3.0
Pothole data	Hugging Face rupesh002/pothole-detection-dataset (MIT listing; a Roboflow export, original terms unverified) and Ryukijano/Pothole-detection-Yolov8 (Roboflow potholes-detection-d4rma v1, CC BY 4.0, attribution required)
Not in the repository	Raw datasets, training checkpoints, the downloaded people model, and phone numbers (read from an environment variable)